

Time-Series Analysis and Forecasting

A Guide to the Concepts Behind

Bitcoin Volatility Prediction

Capstone Project Companion Document

Plain-language explanations, then the mathematics

Contents

1	What Is Time-Series Analysis?	2
1.1	In Plain Terms	2
1.2	The Technical View	2
1.3	Stationarity: The Assumption Behind Most Methods	2
1.4	The Rule That Matters Most: No Lookahead	3
2	What This Project Is About	4
2.1	In Plain Terms	4
2.2	The Technical Definition	4
2.3	Why This Is a Regression Problem	4
2.4	The Defining Property of Volatility: Clustering	4
3	The Models: How and Why	6
3.1	Why Not Random Forest or XGBoost?	6
3.2	Baseline 1: Naive Persistence	6
3.3	Baseline 2: Holt–Winters Exponential Smoothing	6
3.4	Model 1: The GARCH Family	8
3.5	Model 2: HAR-RV and HAR-RV-X	9
3.6	Model 3: LSTM	10
4	How the Models Compare	11
4.1	How Performance Is Measured	11
4.2	Two Kinds of Validation	11
4.3	What a Realistic Result Looks Like	11
5	Glossary	12

1. What Is Time-Series Analysis?

1.1 In Plain Terms

Most datasets are a collection of independent things: a table of customers, a folder of images, a list of houses with their prices. You could shuffle the rows and nothing would be lost — customer #500 has no special relationship with customer #501.

A **time series** is different. It is a sequence of measurements taken on the *same thing* at successive points in time — Bitcoin’s price every hour, a city’s rainfall every day, a company’s revenue every quarter. Here the order is the whole point. Yesterday’s value tells you something about today’s. Shuffle the rows and you destroy the information.

The core idea: in a time series, each observation is connected to the ones before it. Analysis means understanding those connections; forecasting means using them to say something about what comes next.

Time-series analysis is the study of that structure — finding the trend, the repeating cycles, the way shocks fade. **Forecasting** is using that understood structure to predict future values.

1.2 The Technical View

Formally, a time series is an ordered sequence of random variables indexed by time:

$$\{y_t\}_{t=1}^T = y_1, y_2, \dots, y_T$$

The property that makes it a time series rather than a plain dataset is **autocorrelation** — correlation of the series with a lagged copy of itself. The autocorrelation at lag k is:

$$\rho_k = \frac{\text{Cov}(y_t, y_{t-k})}{\sqrt{\text{Var}(y_t) \text{Var}(y_{t-k})}} = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

If $\rho_k \approx 0$ for all k , the observations really are independent and ordinary machine learning applies. If ρ_k is substantially non-zero, there is exploitable temporal structure — and, critically, the standard assumption of independent observations is violated.

A time series is classically decomposed into components:

$$y_t = \underbrace{T_t}_{\text{trend}} + \underbrace{S_t}_{\text{seasonality}} + \underbrace{\varepsilon_t}_{\text{remainder}}$$

1.3 Stationarity: The Assumption Behind Most Methods

Most classical time-series methods require the series to be **stationary** — its statistical properties do not drift over time. Formally, weak stationarity requires:

$$\begin{aligned}\mathbb{E}[y_t] &= \mu && \text{(constant mean)} \\ \text{Var}(y_t) &= \sigma^2 && \text{(constant variance)} \\ \text{Cov}(y_t, y_{t-k}) &= \gamma_k && \text{(autocovariance depends only on the lag } k\text{)}\end{aligned}$$

Prices are almost never stationary (they wander). Returns usually are much closer to it, which is why financial work is nearly always done on returns rather than raw prices. Stationarity is tested

with the **Augmented Dickey–Fuller (ADF) test**, which tests the null hypothesis that a unit root is present (non-stationary):

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^p \delta_i \Delta y_{t-i} + \varepsilon_t$$

A small p -value rejects the null, indicating stationarity.

1.4 The Rule That Matters Most: No Lookahead

Because the data is ordered, the cardinal sin of time-series modelling is **lookahead bias** (or data leakage): letting the model see information that would not have been available at prediction time.

This has two practical consequences that govern every design decision in this project:

1. **Features must be lagged.** To predict the value at time t , the model may only use information from $t - 1$ and earlier.
2. **Splits must respect time.** The training set must come entirely *before* the test set. A random shuffle would let the model train on rows sitting between its own test rows — inflating the score while learning nothing generalisable.

This is why this project uses a strict chronological 80/20 split and `TimeSeriesSplit` for cross-validation, never `train_test_split` with shuffling. It is also the single most common flaw in amateur crypto-prediction projects.

2. What This Project Is About

2.1 In Plain Terms

Most people who model Bitcoin try to predict *where the price is going*. This project deliberately does not. Instead it predicts **volatility** — *how much* the price is likely to move, in either direction.

An analogy: forecasting price is like predicting exactly where a boat will be tomorrow. Forecasting volatility is like predicting how rough the sea will be. The second is a different question, it is more tractable, and for anyone managing risk it is often the more useful one.

The question this project answers: given Bitcoin’s recent hourly price and volume history, how turbulent will the next stretch of trading be?

Volatility matters because it is the direct input to real financial decisions:

- **Option pricing.** The Black–Scholes formula takes volatility (σ) as a required input.
- **Risk limits (Value-at-Risk).** How much could we lose on a bad day? Scales directly with σ .
- **Position sizing.** Trade smaller when the market is choppy, larger when it is calm.
- **Margin requirements.** Exchanges raise collateral requirements when volatility rises.

2.2 The Technical Definition

Let P_t be Bitcoin’s closing price for hour t . The hourly return is:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad \text{or, in log form,} \quad \ell_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log returns are preferred in finance because they are additive across time: the return over n periods is simply $\sum \ell_t$.

The **target variable** of this project is **realised volatility** — the rolling standard deviation of returns over a window of $W = 168$ hours (7 days):

$$\sigma_t = \sqrt{\frac{1}{W-1} \sum_{i=0}^{W-1} (r_{t-i} - \bar{r}_t)^2} \quad \text{where} \quad \bar{r}_t = \frac{1}{W} \sum_{i=0}^{W-1} r_{t-i}$$

Because the data is hourly, this value updates every hour, sliding one hour forward each time. The model always answers: *as of right now, how volatile has the past week been — and what comes next?*

2.3 Why This Is a Regression Problem

A tempting alternative is to classify volatility into “high” and “low” regimes. This project rejects that framing, for a concrete reason: every downstream consumer of a volatility number needs the *number*. Black–Scholes cannot take “high” as an input for σ ; a position-sizing rule cannot divide by “low”. Categorising the target would discard exactly the information that makes it useful.

2.4 The Defining Property of Volatility: Clustering

The single most important empirical fact about financial volatility, first documented by Mandelbrot in 1963, is **volatility clustering**: large moves tend to be followed by large moves, and calm by calm.

In the language of Section 1, this means volatility is strongly *autocorrelated* — ρ_k stays significantly positive for many lags. This is not a nuisance; it is the entire reason forecasting volatility is possible at all. Every model in this project exists to exploit it, and they differ mainly in *how* they represent it.

3. The Models: How and Why

Three models are used, plus two simple baselines. They were chosen on one principle: each must be **time-series-native** — meaning time is built into the model’s own structure, not merely supplied to it as hand-engineered columns.

3.1 Why Not Random Forest or XGBoost?

Generic machine-learning models — linear regression, random forests, gradient boosting — have no internal notion of time. To such a model, each row is an unordered bag of numbers; it cannot tell a time series from a customer database. It “sees” time only through whatever lagged columns the analyst manually creates.

That approach can work well in practice, but it is fundamentally *feature engineering doing the temporal work*, not the model. Since this project is framed as a quantitative-finance time-series study, the models below were chosen because their mathematics is temporal by construction.

3.2 Baseline 1: Naive Persistence

Plain terms: predict that the next value equals the current one. “Tomorrow will be like today.”

The mathematics:

$$\hat{\sigma}_{t+1} = \sigma_t$$

Why it is here: because volatility clusters, this trivial rule is genuinely hard to beat. It sets the floor: any model that cannot beat it has added complexity without adding value. Reporting it honestly is a mark of a serious study.

3.3 Baseline 2: Holt–Winters Exponential Smoothing

Plain terms: forecast using a weighted average of the past, where recent observations count more and older ones fade away exponentially. Holt–Winters adds two refinements: a *trend* (is the level drifting up or down?) and a *seasonal* cycle (does the same pattern repeat every 24 hours?).

The mathematics. The simplest form, Simple Exponential Smoothing, is a one-parameter recursion:

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t, \quad 0 < \alpha < 1$$

Expanding this recursion reveals the exponentially decaying weights explicitly:

$$\hat{y}_{t+1} = \alpha \sum_{j=0}^{\infty} (1 - \alpha)^j y_{t-j}$$

Holt–Winters (additive form) extends this to three interacting components — level ℓ_t , trend b_t , and seasonal s_t with period m :

$$\begin{aligned} \ell_t &= \alpha (y_t - s_{t-m}) + (1 - \alpha) (\ell_{t-1} + b_{t-1}) && \text{(level)} \\ b_t &= \beta (\ell_t - \ell_{t-1}) + (1 - \beta) b_{t-1} && \text{(trend)} \\ s_t &= \gamma (y_t - \ell_t) + (1 - \gamma) s_{t-m} && \text{(seasonal)} \\ \hat{y}_{t+h} &= \ell_t + h b_t + s_{t+h-m} && \text{(forecast)} \end{aligned}$$

Why it is here, and why only as a baseline: exploratory analysis found that volatility varies by hour of day (trading-session overlaps between Asia, Europe, and the US). Holt–Winters, with $m = 24$, is the natural test of whether that seasonality is genuinely *exploitable* for forecasting. It is a baseline rather than a headline model because it *smooths* the level of past volatility; it has

no concept of a market shock and no mechanism describing how volatility *responds* to one. If it beats naive persistence, the daily cycle is real and useful — if not, that is a clean negative result worth reporting.

3.4 Model 1: The GARCH Family

Plain terms: GARCH is the model built specifically to answer this project’s question. Its logic is intuitive: today’s turbulence comes from three sources — a long-run baseline level, an echo of yesterday’s turbulence, and a reaction to how surprising yesterday’s price move was.

The name. *Generalised AutoRegressive Conditional Heteroskedasticity*:

- *Heteroskedasticity* — variance that changes over time (this is a model of volatility);
- *Conditional* — today’s variance is conditioned on yesterday’s information;
- *AutoRegressive* — variance depends on its own past values;
- *Generalised* — extends Engle’s earlier ARCH model.

The mathematics. The return is modelled as a scaled random shock, and the variance follows its own recursion. GARCH(1,1):

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \sim \mathcal{N}(0, 1)$$

$$\sigma_t^2 = \underbrace{\omega}_{\text{baseline}} + \underbrace{\alpha \varepsilon_{t-1}^2}_{\text{reaction to shock}} + \underbrace{\beta \sigma_{t-1}^2}_{\text{persistence}}$$

With just three parameters, $\alpha + \beta$ measures *persistence* — how long a burst of volatility lingers. Values close to 1 (typical in crypto) mean shocks decay very slowly, which is volatility clustering expressed as a single number. The long-run average variance is $\omega/(1 - \alpha - \beta)$.

The improvement: capturing asymmetry. Plain GARCH treats a -5% move and a $+5\%$ move identically, since the shock enters squared. Reality disagrees: bad news raises volatility more than equally-sized good news — the **leverage effect**. Two standard extensions fix this, and this project selects between them by AIC:

GJR-GARCH adds a term that activates only for negative shocks, where $I[\cdot]$ is an indicator function equal to 1 when the condition holds:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \gamma \varepsilon_{t-1}^2 I[\varepsilon_{t-1} < 0] + \beta \sigma_{t-1}^2$$

EGARCH models log-variance, guaranteeing positivity without parameter constraints, and captures asymmetry via the signed term:

$$\ln \sigma_t^2 = \omega + \alpha \left(\left| \frac{\varepsilon_{t-1}}{\sigma_{t-1}} \right| - \mathbb{E} \left| \frac{\varepsilon_{t-1}}{\sigma_{t-1}} \right| \right) + \gamma \frac{\varepsilon_{t-1}}{\sigma_{t-1}} + \beta \ln \sigma_{t-1}^2$$

Why this counts as a genuine improvement: moving from GARCH to EGARCH/GJR is a change to the *equation*, not a wider hyperparameter search. It adds a mechanism the baseline structurally could not express.

A note on model selection. These are fitted by maximum likelihood, not by minimising a prediction loss, so selection uses the **Akaike Information Criterion** rather than a validation score:

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$

where k is the number of parameters and $\hat{L}$ the maximised likelihood — rewarding fit, penalising complexity.

3.5 Model 2: HAR-RV and HAR-RV-X

Plain terms: different market participants operate on different horizons — day traders react to the last few hours, portfolio managers to the last month. The HAR model captures this by predicting volatility from volatility measured over three different time-scales at once: the past day, the past week, and the past month.

The mathematics. Let RV_t denote realised volatility at time t . Define averages over three horizons:

$$RV_t^{(d)} = RV_{t-1}, \quad RV_t^{(w)} = \frac{1}{5} \sum_{i=1}^5 RV_{t-i}, \quad RV_t^{(m)} = \frac{1}{22} \sum_{i=1}^{22} RV_{t-i}$$

(In this project, scaled to hourly data: 24, 24×7 , and 24×30 periods.) The model is then a linear regression on those three regressors:

$$RV_t = \beta_0 + \beta_d RV_t^{(d)} + \beta_w RV_t^{(w)} + \beta_m RV_t^{(m)} + \varepsilon_t$$

Why this is not “just linear regression.” The estimator is ordinary least squares, but the *model* is not generic: its regressors are the target variable itself, measured at three economically meaningful horizons. That structure is the model. Corsi designed it to approximate the long-memory behaviour of volatility — the empirical fact that its autocorrelation decays slowly, like a power law, rather than dying off quickly.

The improvement: HAR-RV-X. The “X” denotes an *exogenous* regressor — an additional predictor from outside the volatility cascade. This project adds the Average True Range, a range-based volatility measure computed from intraday highs and lows:

$$TR_t = \max(H_t - L_t, |H_t - P_{t-1}|, |L_t - P_{t-1}|), \quad ATR_t = \frac{1}{14} \sum_{i=0}^{13} TR_{t-i}$$

$$RV_t = \beta_0 + \beta_d RV_t^{(d)} + \beta_w RV_t^{(w)} + \beta_m RV_t^{(m)} + \beta_x ATR_{t-1} + \varepsilon_t$$

This is a meaningful addition rather than redundancy: RV is built from close-to-close returns, whereas ATR uses the high–low *range*. It therefore captures intraday movement that closing prices miss entirely — genuinely new information, not a restatement of the target. Ridge regularisation is applied, with the penalty strength λ selected by time-series cross-validation:

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \left\{ \sum_t \left(RV_t - \mathbf{x}_t^\top \beta \right)^2 + \lambda \|\beta\|_2^2 \right\}$$

This is worthwhile because the four regressors are themselves correlated (all measure volatility), which makes unregularised coefficients unstable.

3.6 Model 3: LSTM

Plain terms: the two models above impose a fixed formula chosen by the analyst. An LSTM instead *learns* the temporal pattern from the data. It reads the sequence hour by hour, maintaining an internal memory that it updates as it goes — deciding for itself what to remember, what to forget, and what to output.

The mathematics. A Long Short-Term Memory cell maintains a cell state C_t (long-term memory) and a hidden state h_t (working output), governed by three gates. With $\sigma(\cdot)$ the logistic sigmoid and $\odot$ elementwise multiplication:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && \text{forget gate: what to discard} \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && \text{input gate: what to store} \\ \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) && \text{candidate values} \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t && \text{update cell state} \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) && \text{output gate} \\ h_t &= o_t \odot \tanh(C_t) && \text{hidden output} \end{aligned}$$

The essential design feature is the cell-state update: it is *additive* rather than multiplicative. This is what allows gradients to flow backwards across many timesteps without vanishing — the failure that cripples simple recurrent networks and the reason LSTMs can learn long-range dependencies.

Input shape. Unlike the other models, the LSTM consumes a three-dimensional tensor:

$$X \in \mathbb{R}^{n \times L \times p}$$

where n is the number of training samples, L the lookback window (how many past hours it sees at once), and p the number of features per hour. Each sample is a *window* of history, not a single row.

Why it is here: GARCH and HAR both assume a specific functional form. If the real relationship involves interactions or nonlinearities they cannot express — for instance, momentum mattering only when volume is also elevated — an LSTM can in principle discover it. Including it directly tests whether the classical models are leaving structure on the table.

The honest trade-off. LSTMs are data-hungry and overfit noisy targets easily; volatility is about as noisy as targets get. They also forfeit interpretability: GARCH's α and β have precise financial meanings, while an LSTM's thousands of weights have none. This is included as a genuine comparison, not on the assumption that the more complex model wins.

4. How the Models Compare

Model	What it models	How time enters	Main strength / weakness
Naive persistence	Nothing — copies last value	Trivially, one lag	Free and hard to beat / no mechanism
Holt–Winters	Level, trend, seasonality	Exponentially weighted recursion	Tests daily cycle / cannot model shocks
GARCH family	Conditional <i>variance</i>	Built into the variance recursion	Purpose-built, interpretable / rigid form
HAR-RV(-X)	Volatility across 3 horizons	Regressors <i>are</i> lagged volatility	Captures long memory simply / linear only
LSTM	Learned sequence mapping	Reads the raw ordered window	Flexible, nonlinear / opaque, overfits

4.1 How Performance Is Measured

All models are scored on identical held-out data with three standard regression metrics. For n predictions $\hat{y}_i$ against actuals y_i :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

RMSE is the headline metric because squaring penalises large errors disproportionately — and in risk management, badly underestimating a volatility spike is exactly the error that causes real losses. MAE is reported alongside it because it is less distorted by rare extreme values. R^2 gives the share of variance explained, where 0 means no better than predicting the mean.

4.2 Two Kinds of Validation

Retrospective (historical hold-out). The last 20% of the historical data, never seen during training, split strictly chronologically. Hyperparameters are tuned only within the training portion, using `TimeSeriesSplit`, which always trains on earlier folds and validates on later ones:

```
Fold 1:  [train][val]
Fold 2:  [train][train][val]
Fold 3:  [train][train][train][val]
```

Prospective (live). The stronger test. The trained models are saved to disk, and a separate notebook section fetches genuinely new hourly candles — data that did not exist when training finished — and logs a prediction for each. Once 168 hours have elapsed for a given prediction, its true realised volatility becomes computable, and the prediction is scored against reality.

This distinction matters. A historical hold-out can still flatter a model through subtle leakage or through the analyst’s own iteration on the test set. Predictions logged *before* the outcome exists cannot be fooled — there is no way to peek at an answer that has not happened yet.

4.3 What a Realistic Result Looks Like

Volatility is driven substantially by information absent from price history — regulatory announcements, exchange failures, macroeconomic news. No amount of feature engineering recovers what is not in the data, so there is a hard ceiling on achievable accuracy. Modest improvements over naive persistence are the realistic and honest expectation. A model reporting near-perfect accuracy on this task should be treated as evidence of leakage, not of success.

5. Glossary

Autocorrelation

Correlation of a series with a lagged copy of itself; the defining property of time-series data.

Realised volatility

Volatility computed from observed historical returns, as opposed to *implied* volatility, which is backed out of option prices.

Volatility clustering

The empirical tendency for large price moves to be followed by large moves, and calm periods by calm.

Leverage effect

The asymmetry whereby negative price shocks increase volatility more than positive shocks of equal size.

Stationarity

The property that a series' statistical characteristics (mean, variance, autocovariance) do not change over time.

Lookahead bias / leakage

Allowing a model access to information unavailable at prediction time — the central methodological hazard in time-series work.

Long memory

Autocorrelation that decays slowly (hyperbolically) rather than exponentially; characteristic of volatility and the motivation for HAR-RV.

Exogenous regressor

A predictor drawn from outside the model's core recursion — the “X” in HAR-RV-X.

AIC

Akaike Information Criterion, $2k - 2 \ln \hat{L}$; a model-selection score trading goodness-of-fit against parameter count.

References

- Bollerslev, T. (1986). Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- Corsi, F. (2009). A Simple Approximate Long-Memory Model of Realized Volatility. *Journal of Financial Econometrics*, 7(2), 174–196.
- Engle, R. (1982). Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, 50(4), 987–1007.
- Glosten, L., Jagannathan, R., & Runkle, D. (1993). On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks. *Journal of Finance*, 48(5), 1779–1801.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Holt, C. C. (1957). Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages. *ONR Memorandum*, 52.

- Mandelbrot, B. (1963). The Variation of Certain Speculative Prices. *The Journal of Business*, 36(4), 394–419.
- Nelson, D. (1991). Conditional Heteroskedasticity in Asset Returns: A New Approach. *Econometrica*, 59(2), 347–370.
- Winters, P. R. (1960). Forecasting Sales by Exponentially Weighted Moving Averages. *Management Science*, 6(3), 324–342.